🔭 Scope & Closures

How Python looks up variable names

Shwetank's Python Journey • LEGB Rule • global • Closures • Factories

📌 The LEGB Rule — How Python Finds a Name

When Python sees a variable name, it searches in exactly this order — stopping at the first match:

Scope Level	What it means
L — Local	Names assigned inside a function. First place Python looks.
E — Enclosing	Names in the scope of any enclosing function (closures).
G — Global	Names assigned at the top level of the module/script.
B — Built-in	Python's built-in names: print, len, range, etc.

NOTE	Think of LEGB as a set of nested boxes. Python starts at the innermost box (Local) and works outward until it finds the name or raises a NameError.

① Local vs Global Scope — *The username example*

Two variables both named username exist here — but they live in completely separate scopes. The one inside func() is local; the one at the top is global.

```
username = 'shwetank'   # ← GLOBAL scope

def func():
    username = 'chai'   # ← LOCAL scope (new variable, does NOT touch global)
    print(username)     # sees LOCAL username

print(username)  # sees GLOBAL username
func()           # sees LOCAL username
```

▶ **Output**

```
shwetank
chai
```

NOTE Assignment inside a function (username = 'chai') creates a brand-new local variable. It never touches the global one — they just happen to share a name.

? Interview Q: Can a function read a global variable without declaring it?

☑ Yes — reading works fine (Python walks up to Global scope). But assigning to a global name inside a function without global keyword creates a local variable instead, leaving the global unchanged.

② Reading a Global from Inside a Function

func2 doesn't assign to x — it only reads it. Python can't find x locally, so it walks up to the global scope and finds x = 99. No global keyword needed for reading.

```
x = 99              # GLOBAL

def func2(y):
    z = x + y      # x is read from GLOBAL scope
    return z

result = func2(1)
print(result)       # 100
```

▶ Output

```
100
```

TIP Reading globals is fine. The problems start when you try to assign to a global without declaring it — Python treats that as a new local variable, often causing an UnboundLocalError.

③ The global Keyword *— Modifying a global variable*

The global keyword tells Python: 'when I say x inside this function, I mean the global x — not a local one.' This lets you reassign a global from inside a function.

```
x = 99

def func3():
    global x   # now x refers to the GLOBAL x
    x = 88     # this changes the global x
```

```
func3()
print(x)    # 88  ← global was mutated!
```

▶ **Output**

```
88
```

🚫 AVOID	Shwetank's comment in the code ('avoid krna chaiye aisa') is exactly right! Using global makes code hard to reason about — any function can silently change shared state. Prefer passing values as arguments and returning results instead.

❓ Interview Q: What is the problem with global variables?

☑ They create hidden dependencies — any part of the code can change them at any time. This makes bugs hard to track. Pass data through function parameters and return values instead; it keeps functions predictable and testable.

④ Enclosing Scope & Closures *— f1 / f2 example*

When a function is defined inside another function, the inner function can access the outer function's variables — even after the outer function has returned. This is called a closure.

```
def f1():
    x = 88            # x lives in f1's LOCAL scope
    def f2():
        print(x)      # f2 closes over x from ENCLOSING scope (f1)
    return f2         # return the inner function itself

myresult = f1()      # f1 runs → returns f2
myresult()           # calls f2 → still remembers x = 88!
```

▶ **Output**

```
88
```

📝 NOTE	f1() has already finished executing by the time myresult() is called — but f2 still remembers x = 88. This 'memory' is the closure. Python keeps f1's local scope alive as long as f2 references it.

❓ Interview Q: What is a closure in Python?

☑ A closure is an inner function that remembers the variables from its enclosing (outer) function's scope, even after that outer function has finished executing. The inner function 'closes over' those variables.

⑤ Closure Factory / Function Factory *— chaicoder example*

A factory function uses closures to create customised functions on the fly. chaicoder(num) returns a new function that 'bakes in' the value of num — each call produces a different specialised function.

```
def chaicoder(num):
    def actual(x):
        return x ** num   # num is captured from enclosing scope
    return actual

f = chaicoder(2)   # f is now a 'squarer' (num=2 baked in)
g = chaicoder(3)   # g is now a 'cuber'   (num=3 baked in)

print(f(3))   # 3 ** 2 = 9
print(g(3))   # 3 ** 3 = 27
```

▶ **Output**

```
9
27
```

NOTE	f and g are completely independent functions. Each has its own closure keeping its own copy of num. This is a powerful alternative to classes when you only need to carry a small amount of state.

TIP	This pattern is the foundation of decorators in Python. A decorator is essentially a factory that wraps one function with another — using exactly this closure mechanism.

? Interview Q: What is a function factory?

☑ A function that returns different customised functions depending on its arguments. Each returned function closes over the arguments, 'remembering' them permanently. chaicoder(2) and chaicoder(3) produce two different functions from one definition.

Full Code — Annotated Output

Code	Prints / Concept
`print(username) → func()`	shwetank then chai — local vs global
`result = func2(1); print(result)`	100 — reading global x=99 + y=1
`func3(); print(x)`	88 — global modified via global keyword

`myresult = f1();  myresult()`	88 — closure remembers x from f1
`f = chaicoder(2);  print(f(3))`	9 — factory with num=2 baked in
`g = chaicoder(3);  print(g(3))`	27 — factory with num=3 baked in

Scope is Python's GPS — LEGB tells it exactly where to look for every name.